

9. Организация и архитектура на компютърните системи.

Архитектура на фон Нойман.

Компютърни кодове и представяне на информация в компютъра.

1. Организация и архитектура на компютърните системи

Концептуалното изграждане на съвременните изчислителни машини се основава на две фундаментални, но различаващи се категории: **архитектура** и **организация**.

1.1 Разлика между архитектура и организация

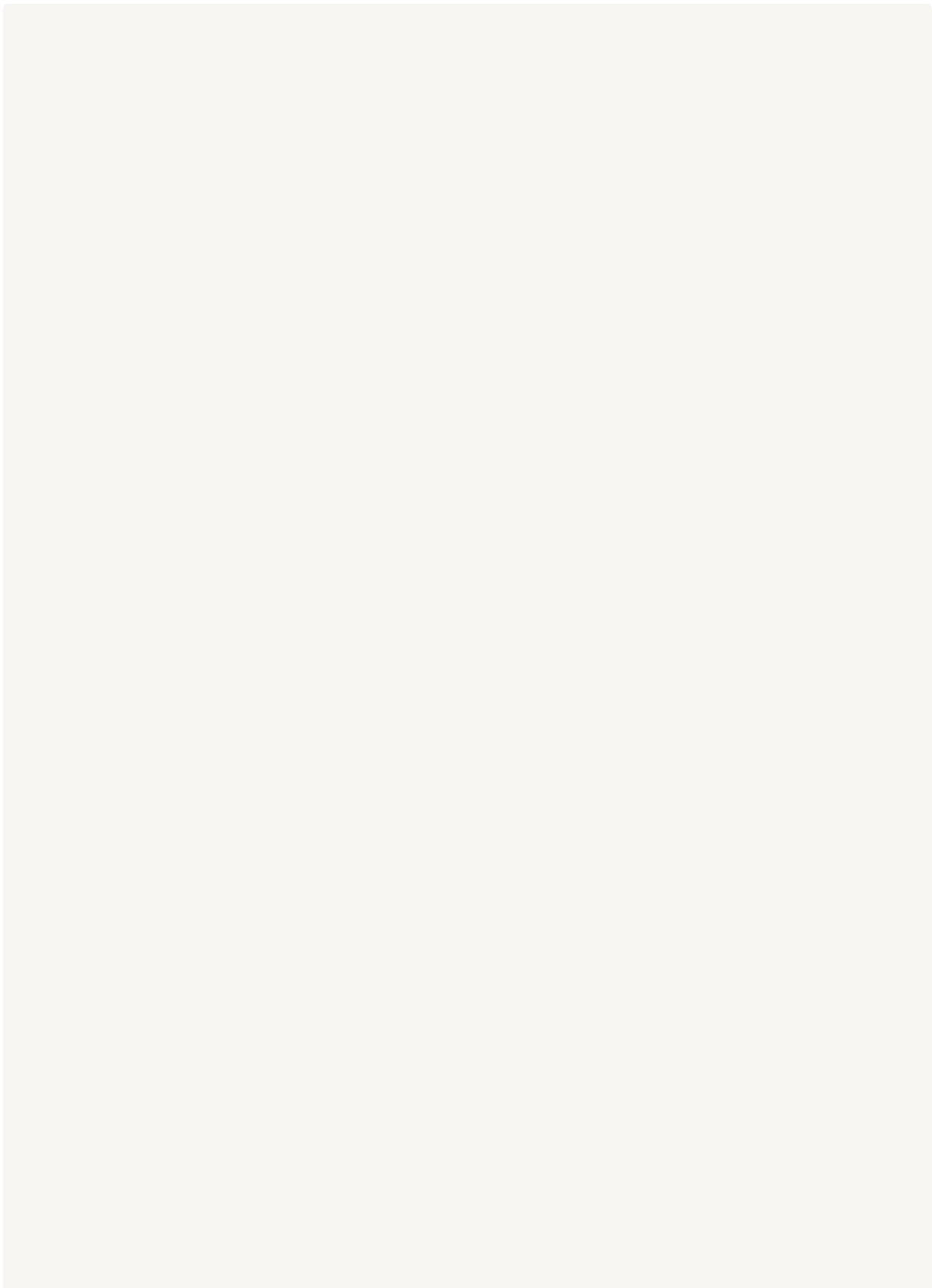
- **Архитектура на компютърната система (АКС):** Определя функционалните характеристики, които са видими за програмиста, включително набора от инструкции, адресното пространство и начина на изпълнение на програмите.
 - Примери: Набор от инструкции (ISA - Instruction Set Architecture), брой и видове регистри, режими на адресиране на паметта, механизми за управление на прекъсванията, разрядност на данните (напр. 32-битова или 64-битова архитектура).
- **Организация на компютърната система (ОКС):** Описва начина, по който хардуерните компоненти са свързани и взаимодействат помежду си.
 - Примери: Конкретни апаратни сигнали за управление, интерфейси между компютъра и периферията, технология на паметта (напр. DDR4).

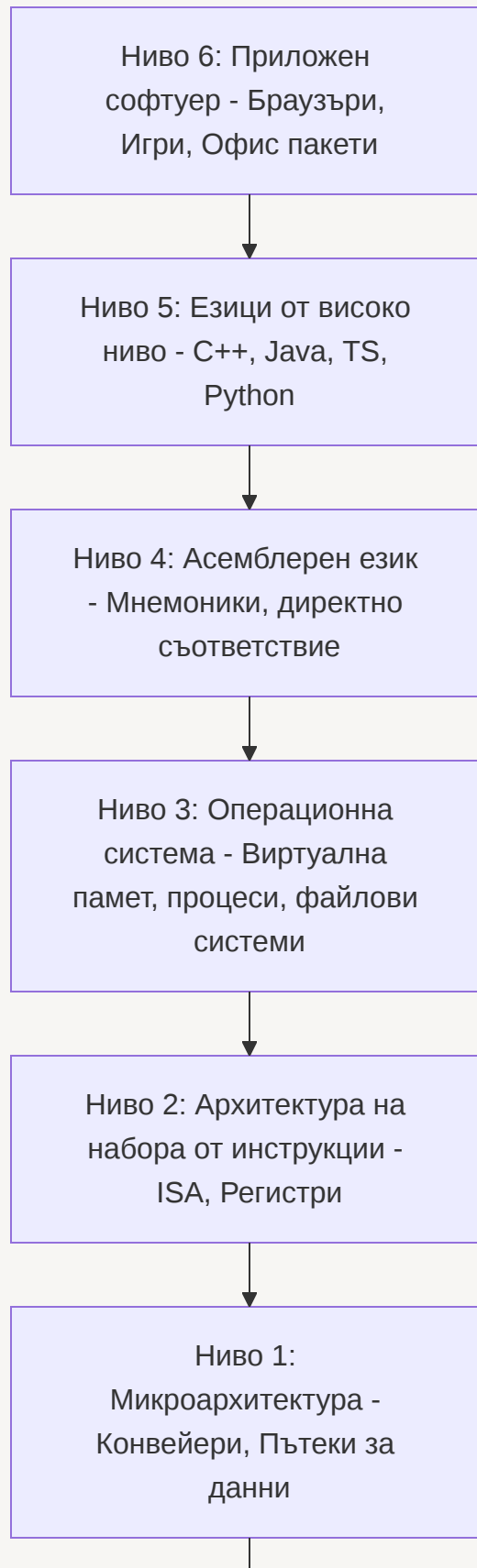
срещу DDR5), използване на конвейерна обработка (pipelining).

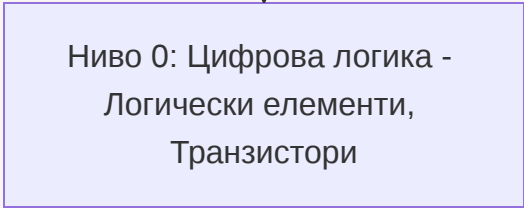
Аналогия: Ако архитектурата дефинира, че една машина трябва да може да умножава две числа (инструкция MUL), то организацията определя дали това ще се извършва от специализиран апаратен умножител или чрез последователност от събирания и измествания на софтуерно/микропрограмно ниво.

1.2 Йерархични нива в компютърните системи

Модерните компютърни системи се разглеждат като многослойни структури, където всяко по-горно ниво абстрахира сложността на по-долното:







Ниво 0: Цифрова логика -
Логически елементи,
Транзистори

Всяко по-високо ниво се изпълнява чрез интерпретация или трансляция (компилиране) към нивото непосредствено под него. Например изходният код на C++ се транслира в асемблерен код, който на свой ред се асемблира в машинни инструкции от ISA нивото, изпълнявани от микроархитектурата.

2. Архитектура на фон Нойман

През 1945 г. математикът Джон фон Нойман (John von Neumann) предлага теоретичен модел за построяване на изчислителна машина, който и до днес стои в основата на почти всички масови микропроцесори.

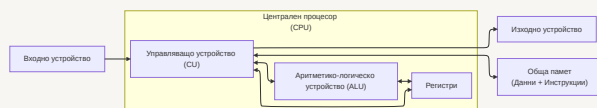
2.1 Основни принципи на фон Нойман

Архитектурата се базира на четири фундаментални принципа:

- 1. Принцип на съхраняваната програма (Stored-Program Concept):**
Инструкциите на програмата и данните, с които те оперират, се съхраняват заедно в едно и също споделено адресируемо адресно пространство (обща памет). Компютърът не се пренастройва физически за всяка нова задача.
- 2. Принцип на последователното изпълнение:** Инструкциите се извличат от паметта и се изпълняват последователно (една след друга) по подразбиране, освен ако не е подадена инструкция за условен или безусловен преход.
- 3. Принцип на линейната организация на паметта:** Паметта се състои от едномерен масив от клетки (байтове/думи), всяка от които притежава уникален числен адрес.
- 4. Използване на двоична бройна система:** Всички данни и инструкции се кодират в двоичен вид (0 и 1).

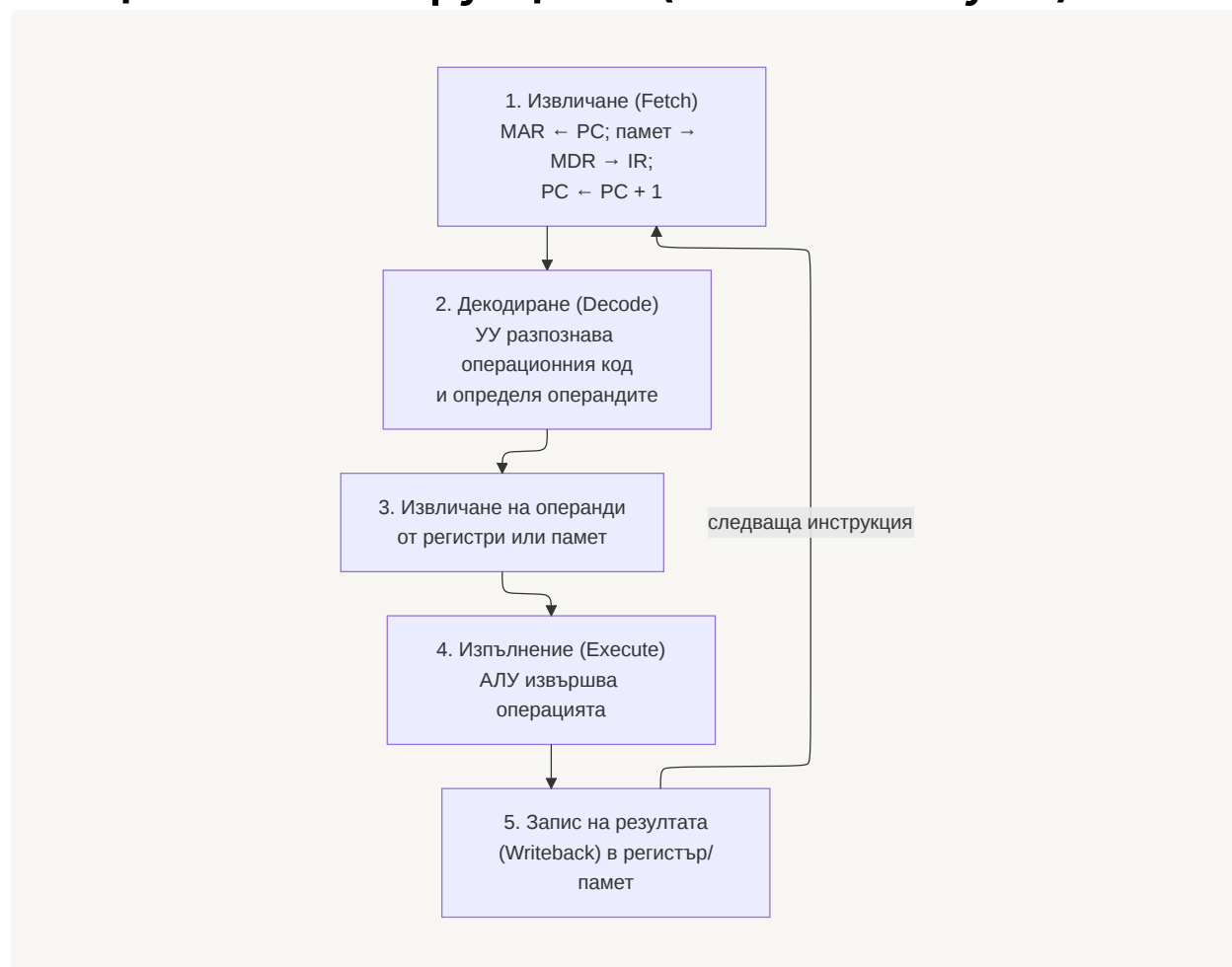
2.2 Блокова структура и компоненти

Моделът на фон Нойман се състои от пет основни функционални блока:



- **Аритметико-логическо устройство (ALU):** Извършва аритметични (събиране, изваждане) и логически (**AND**, **OR**, **NOT**, **XOR**) операции над данни.
- **Управляващо устройство (CU - Control Unit):** Координира работата на всички останали компоненти. То декодира инструкциите, извлечени от паметта, и генерира управляващи сигнали към ALU, паметта и I/O устройствата.
- **Регистри:** Свърхбързи локални клетки от памет в самия процесор. Ключови сред тях са:
 - **PC (Program Counter):** Пази адреса на следващата инструкция за изпълнение.
 - **IR (Instruction Register):** Пази текущо изпълняваната инструкция.
 - **MAR (Memory Address Register):** Пази адреса в паметта, до който ще се осъществи достъп за четене или запис.
 - **MBR/MDR (Memory Buffer/Data Register):** Пази стойността на данните, прочетени от или подготвени за запис в паметта.
 - **AC (Accumulator):** Регистър за временно съхранение на резултатите от ALU.
- **Памет (Memory):** Съхранява както текущо изпълняваната програма (инструкции), така и входните, междинните и крайните резултати (данни).
- **Входно/Изходни устройства (I/O):** Входните устройства трансформират външна информация в цифров вид (напр. клавиатура, мишка), а изходните преобразуват цифровите данни в достъпна за външния свят форма (напр. монитор, принтер).

2.3 Цикъл на инструкцията (Instruction Cycle)



2.4 Тясното място на фон Нойман (Von Neumann Bottleneck)

Тъй като и данните, и инструкциите преминават през една и съща физическа системна шина, процесорът не може едновременно да чете инструкцията и да чете/записва данни в паметта. Този физически лимит води до изчакване на процесора и се нарича "Тясното място на фон Нойман".

Алтернатива е **Харвардската архитектура**, при която инструкциите и данните се съхраняват в отделни паметни с отделни шини, което позволява паралелен достъп:

Критерий	Архитектура на фон Нойман	Харвардска архитектура
Памет	Обща за код и данни	Отделни паметни за код и данни

Критерий	Архитектура на фон Нойман	Харвардска архитектура
Шина	Една обща шина	Отделни шини
Скорост	По-бавна (споделян достъп)	По-бърза (паралелен достъп)
Гъвкавост	По-гъвкава (динамично преразпределение)	По-ограничена, но по-предвидима
Приложение	Персонални компютри, сървъри	Микроконтролери, DSP, вградени системи

На практика повечето съвременни процесори използват **модифицирана Харвардска архитектура** - на ниво кеш паметта е разделена (L1 instruction cache / L1 data cache), докато основната оперативна памет остава единна, комбинирайки предимствата на двата подхода.

3. Компютърни кодове и представяне на информация в компютъра

На физическо ниво информацията в компютъра се представя чрез сигнали с две устойчиви състояния, съответстващи на логическа 0 (ниско напрежение) и логическа 1 (високо напрежение). Това налага използването на двоичната бройна система.

3.1 Позиционна бройна система и преобразувания

Едно число A в бройна система с основа q се представя във вида:

$$A_{(q)} = a_n q^n + a_{n-1} q^{n-1} + \dots + a_1 q^1 + a_0 q^0 + a_{-1} q^{-1} + \dots + a_{-m} q^{-m}$$

Най-използваните системи в компютърните науки са:

- Двоична (Binary), $q = 2$: Използва цифрите $\{0, 1\}$.
- Шестнадесетична (Hexadecimal), $q = 16$: Използва $\{0..9, A, B, C, D, E, F\}$, където $A = 10, \dots, F = 15$. Тя служи за компактно представяне на двоични низове (всеки 4 двоични бита = 1 шестнадесетичен знак).

Преобразуване от десетична в двоична система

- **За цяла част:** Извършва се последователно делене на основата 2 с отделяне на остатъците в обратен ред.
- **За дробна част:** Извършва се последователно умножение по основата 2 и отделяне на цялата част.

3.2 Представяне на цели числа със знак

За представяне на цели числа в компютъра се заделя фиксиран брой битове ($n = 8, 16, 32, 64$). Най-старшият бит (MSB - Most Significant Bit) се заделя за знак: 0 за положително, 1 за отрицателно число.

Използват се три основни компютърни кода:

3.2.1 Прав код (Sign-Magnitude)

Най-старшият бит определя знака, а останалите $n - 1$ бита съдържат абсолютната стойност (модула) на числото.

- *Пример за +5 в 8 бита:* 0000 0101
- *Пример за -5 в 8 бита:* 1000 0101
- *Недостатък:* Съществуват две представяния на нулата (00000000 за +0 и 10000000 за -0), а хардуерната реализация на аритметичните операции е сложна.

3.2.2 Обратен код (Ones' Complement)

При положителните числа съвпада с правия код. За отрицателните числа се извършва инвертиране (побитово отрицание NOT) на всички битове на съответното положително число (включително знаковия бит).

- *Пример за +5 в 8 бита:* 0000 0101
- *Пример за -5 в 8 бита:* 1111 1010
- *Недостатък:* Отново има две нули (00000000 и 11111111).

3.2.3 Допълнителен код (Two's Complement) - Стандарт в компютрите

Положителните числа съвпадат с правия код. За отрицателните числа алгоритъмът е: **Обратен код + 1**.

Пример за -5 в 8 бита:

1. +5 в прав код:

```
0000 0101
```

2. Инвертиране (обратен код):

```
1111 1010
```

3. Добавяне на 1:

```
1111 1010 + 1 = 1111 1011
```

Предимства: Нулата има уникално представяне (00000000). Операцията изваждане се свежда до събиране: $A - B = A + (-B)_{\text{доп.код}}$. Хардуерът се нуждае само от суматори.

3.3 Представяне на реални числа с плаваща запетая

За представяне на реални числа (дробни числа с много голям или много малък диапазон) се използва стандартът IEEE 754. Числото се представя в кодиран научно-нормализиран вид:

$$A = (-1)^s \times M \times q^p$$

Където s е знаков бит, M е мантиса, а p е експонента (порядък).

3.3.1 Стандартни формати на IEEE 754:

Единична точност (Single Precision) - 32 бита (тип `float`):

- 1 бит за знак (s)

- 8 бита за експонента (e), кодирана с отместване (bias) +127
- 23 бита за мантиса (m), където първата единица пред запетаята е скрита (1.xxxx)

Двойна точност (Double Precision) - 64 бита (тип double):

- 1 бит за знак (s)
- 11 бита за експонента (e) с отместване +1023
- 52 бита за мантиса (m)

3.4 Кодиране на текстова и нечислова информация

За да се обработва текст, всеки символ трябва да съответства на конкретно цяло число (код).

- **ASCII (American Standard Code for Information Interchange):** Използва 7 бита (общо 128 символа), разширените версии ползват 8 бита (256 символа). Включва латинската азбука, цифри, пунктуационни знаци и управляващи кодове. Не е достатъчен за интернационални нужди.
- **Unicode:** Универсален стандарт, който заделя уникален код за всеки символ от всички писмености в света (включително емоджита и исторически йероглифи). Най-разпространената кодировка е **UTF-8** - кодиране с променлива дължина (от 1 до 4 байта), което е напълно съвместимо с ASCII за първите 128 символа и оптимизира пространството в мрежови комуникации.